

24CSEN2021

**COMPUTER ORGANIZATION AND
ARCHITECTURE**

UNIT-I

Textbooks

1. C. Hamacher, Z. Vranesic and S. Zaky, "Computer Organization", 5th edition, McGrawHill, 2017
2. W. Stallings, "Computer Organization and Architecture -Designing for Performance", 11th edition, Prentice Hall of India,2022

• References:

1. D. A. Patterson and J. L. Hennessy, "Computer Organization and Design The Hardware/Software Interface", 1998
2. J .P. Hayes, "Computer Architecture and Organization", 1998
- 3.,https://onlinecourses.nptel.ac.in/noc21_cs61/preview

Overview of Computer Architecture and Organization

- Contrast between computer architecture and organization
- Fundamentals of computer architecture
- Organization of von Neumann machine
- CPU
- Memory
- Input-output subsystems
- Control unit
- Instruction set architecture of a CPU—registers
- Instruction execution cycle
- RTL interpretation of instructions
- Addressing modes
- Types of instructions
- Instruction set
- CISC & RISC
- Big-endian & Little-endian representation

Computer Architecture and Organization

- Computer Architecture and Organization are fundamental concepts in designing computer systems, each focusing on different aspects of system functionality and implementation.
- Computer Architecture refers to the attributes of a computer system that are visible to the user.
- This includes elements like addressing techniques, instruction sets, and the representation of data in bits.
- These attributes have a direct impact on how programs execute, as they define the abstract system.
- Essentially, computer architecture addresses the question of what the system does, providing a framework for understanding the capabilities and functionalities that users can expect.

Basic Components of a Computer System

1. Motherboard

➤ The main circuit board connects all components. It determines the overall layout, size, and compatibility of hardware components like the CPU, RAM, and GPU.

2. Central Processing Unit (CPU)

➤ It is often called the brain of the computer; the CPU executes instructions and processes data.

3. Graphics Processing Unit (GPU)

➤ This component is responsible for rendering images, animations, and video.

4. Random Access Memory (RAM)

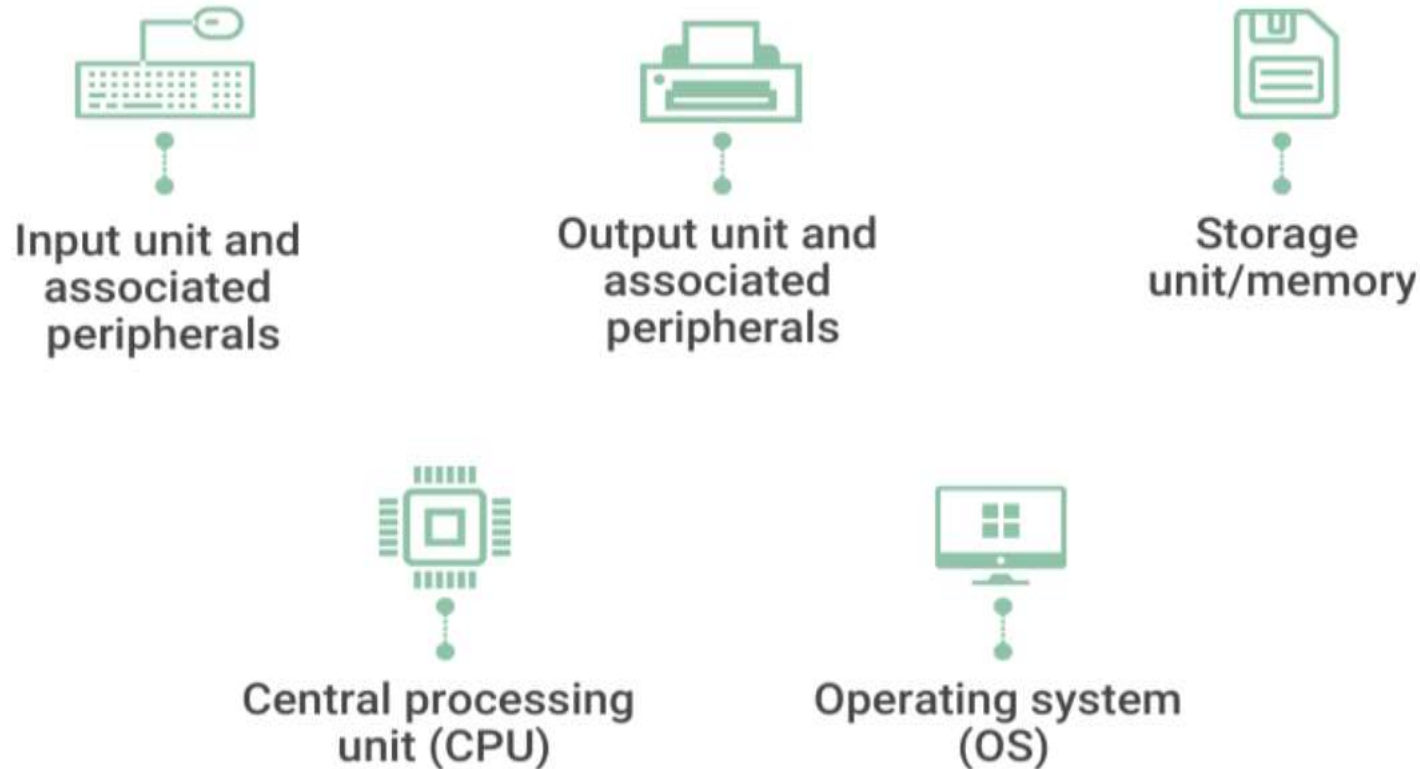
➤ RAM temporarily stores data that the CPU uses for active tasks. More RAM can improve performance, especially for multitasking.

5. Storage Device

➤ This is where all your data, programs, and the operating system are stored. Common types include Hard Disk Drives (HDDs) and Solid-State Drives (SSDs).

Fundamentals of computer architecture

- The main components of a computer architecture are the CPU, memory, and peripherals.
- All these elements are linked by the system bus, which comprises an address bus, a data bus, and a control bus.



Fundamentals of computer architecture

Purpose of Basic Computer Architecture: Yes, almost everything that happens in a system is related to the computer handling some kind of numeric data processing-understanding that corresponds to surfing the web or printing something out.

Data in numbers: A numeral is the way any information is stored by a computer.

Manipulating data: All management of information on computers is carried out through numerical processing.

Multifaceted functions: Basic computer architecture components are both soft and hard. Any basic computer has at least a processor, which is hardware that executes the computer programs.

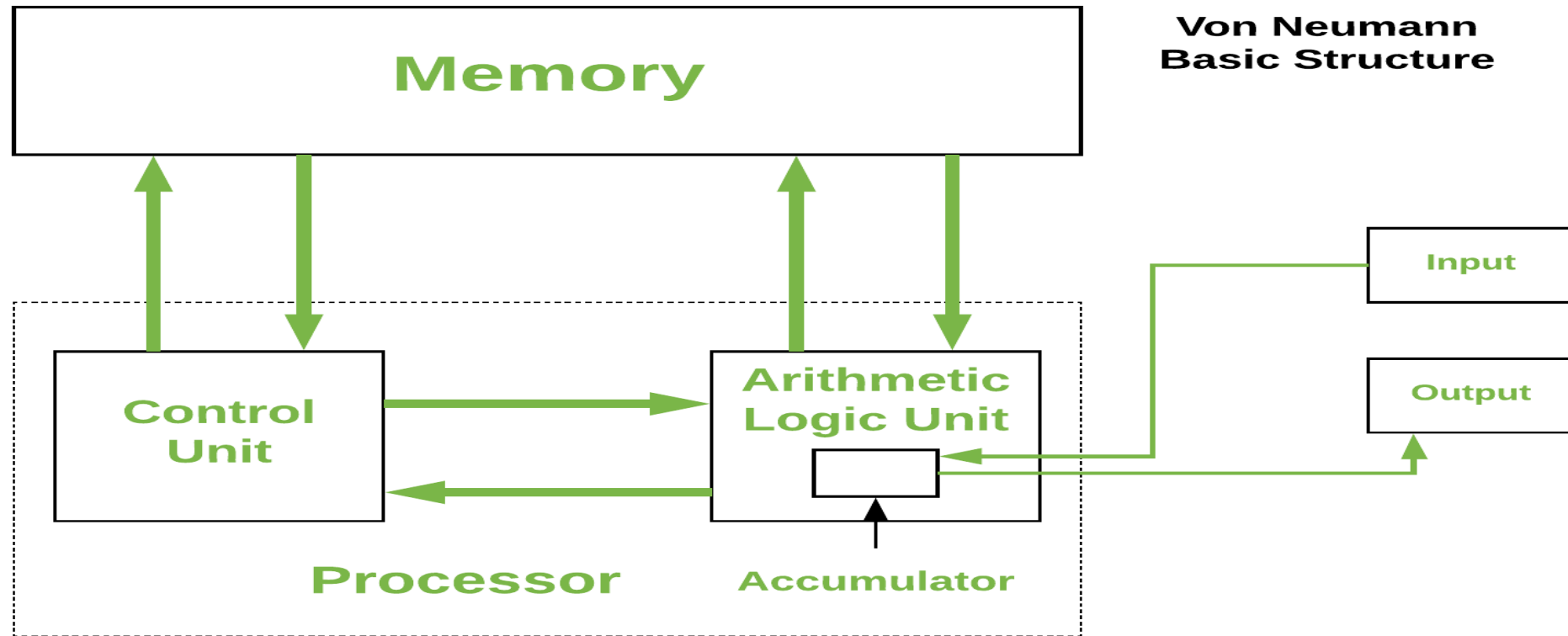
Booting up: At the most elementary level of computer design, while the computer is switched on, programs executed by the processor of a computer will perform the configuration of the proper functioning of such a computer, known as firmware, and initialize the various component hardware into a known state.

Temporary storage capacity: Memory is an essential component of computer architecture, with several types commonly being found in one system.

Organization of von Neumann machine

- One of the most important models in this field is the Von Neumann architecture, which is the foundation of most modern computers. Named after John von Neumann, this architecture introduced the concept of storing both data and instructions in the same memory.
- There have been 2 types of Computers:
 - **Fixed Program Computers** - Their function is very specific and they couldn't be reprogrammed, e.g. Calculators.
 - **Stored Program Computers** - These can be programmed to carry out many different tasks; applications are stored on them, hence the name.
- The Von Neumann architecture popularized the stored-program concept, making computers more flexible and easier to reprogram. This design stores both data and instructions in the same memory, simplifying hardware design and enabling general-purpose computing.

Organization of von Neumann machine

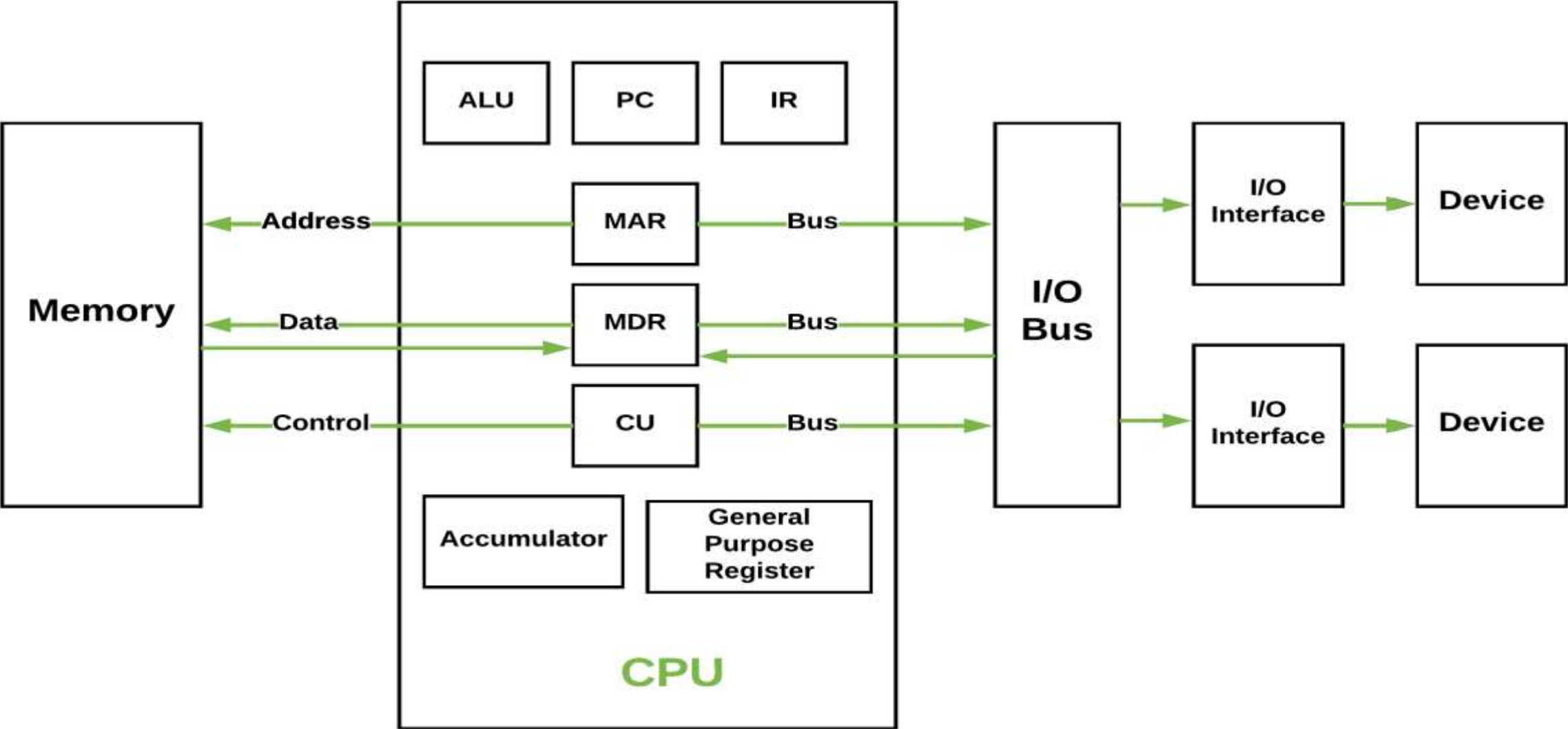


➤ The structure described in the figure outlines the basic components of a computer system, particularly focusing on the memory and processor.

Organization of von Neumann machine

- **Memory:** This is where data and instructions are stored. It is a crucial part of the computer system that allows for the storage and retrieval of information.
- **Control Unit:** This component manages the operations of the computer. It directs the flow of data between the CPU and other components.
- **Arithmetic Logic Unit (ALU):** The ALU performs arithmetic and logical operations. It is responsible for calculations and decision-making processes.
- **Input:** This refers to the devices or methods through which data is entered into the computer system.
- **Output:** This refers to the devices or methods through which data is presented to the user or other systems.
- **Processor:** The processor, or CPU, is the central component that carries out the instructions of a computer program. It includes the ALU and Control Unit.
- **Accumulator:** This is a register in the CPU that stores intermediate results of arithmetic and logic operations.

Basic CPU structure, illustrating ALU

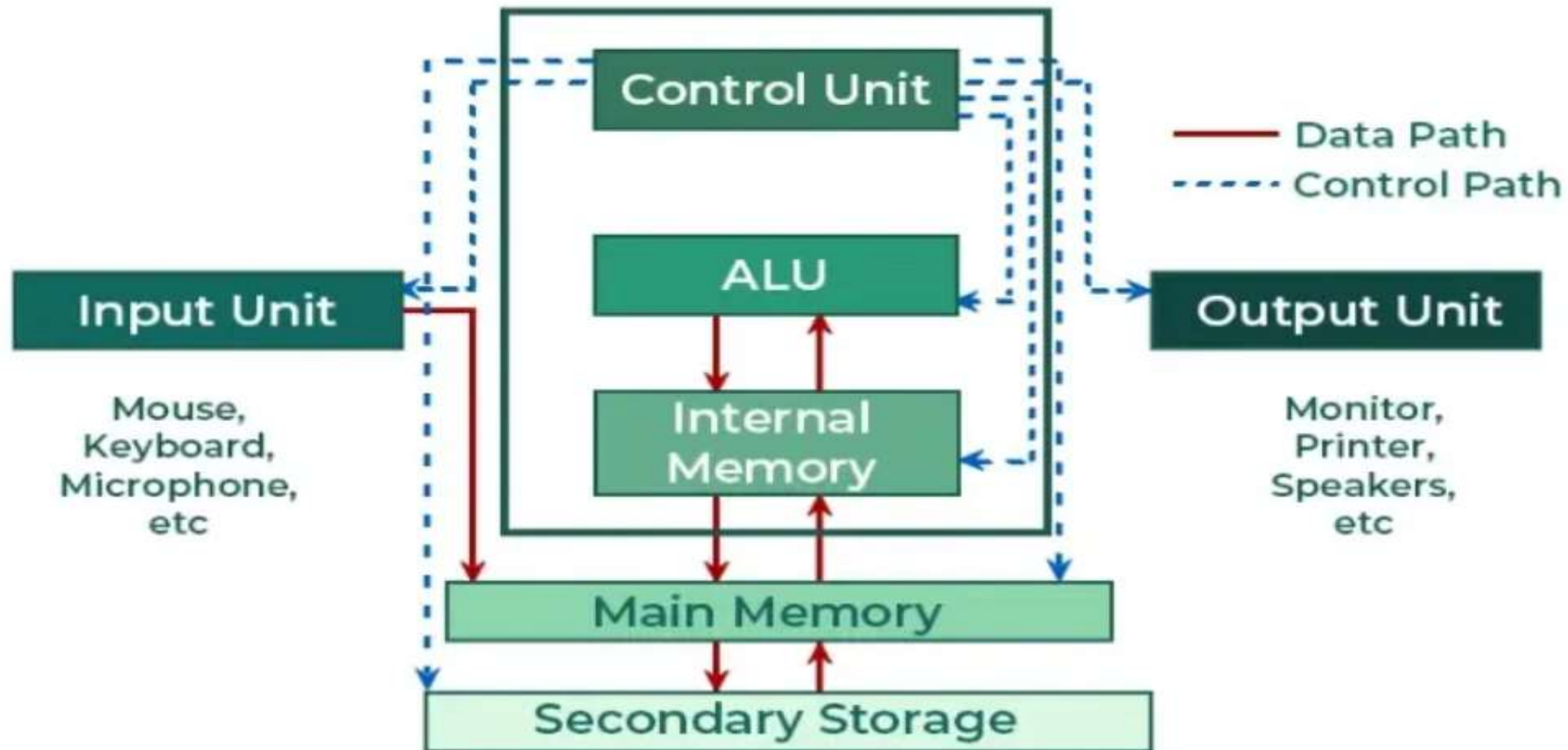


CPU (Central Processing Unit)

- A Central Processing Unit (CPU) is the "brain" of a computer, responsible for executing instructions and performing calculations.
- It interprets and executes commands from software programs, controlling other components within the machine.
- Essentially, it's the primary processor that handles all the core functions of a computer.
- **Control Unit (CU):** It controls the CPU's operations by reading and following instructions. It also manages the flow of data inside the CPU.
- **ALU (Arithmetic Logic Unit):** It does all the math and logic calculations, like addition, subtraction, and comparisons (such as checking if two numbers are equal).
- **Input Unit:** This part gets data from devices like a keyboard, mouse, or microphone, and sends it to the CPU to be processed.
- **Output Unit:** After the CPU processes the data, the output unit sends it to devices like a monitor, printer, or speakers so the user can see or hear the result.
- **Internal Memory:** This stores temporary data and instructions that the CPU needs while it's working, like the registers and cache memory.
- **Main Memory:** It holds the data and instructions the CPU is currently using, often in RAM (Random Access Memory).
- **Secondary Storage:** This is where data and programs are stored when they're not being used right away, like on hard drives or SSDs.

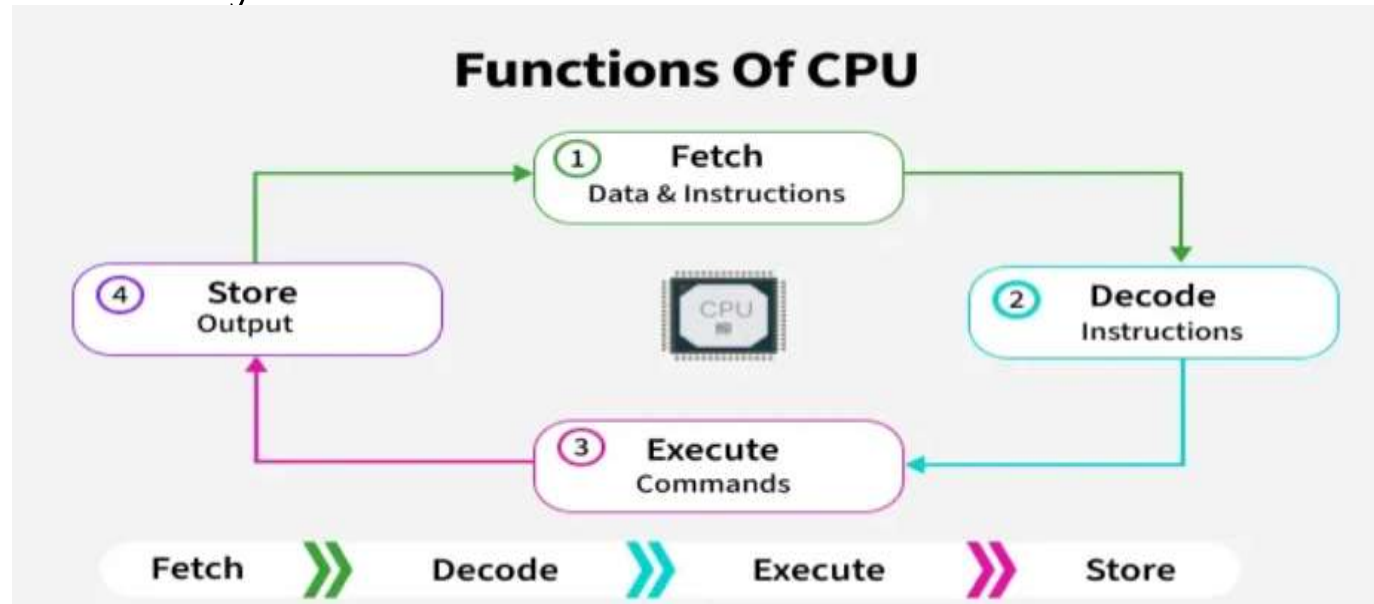
Components of CPU

Components of CPU



Functions of the CPU

➤ The CPU's main job is to process instructions from programs. It does this through a process called the **Fetch-Decode-Execute-Store** cycle:



Types of CPUs

CPUs come in different types, depending on how many cores they have. A core is like a mini-CPU inside the main CPU, and more cores mean the CPU can do more tasks at once. Here are the main types:

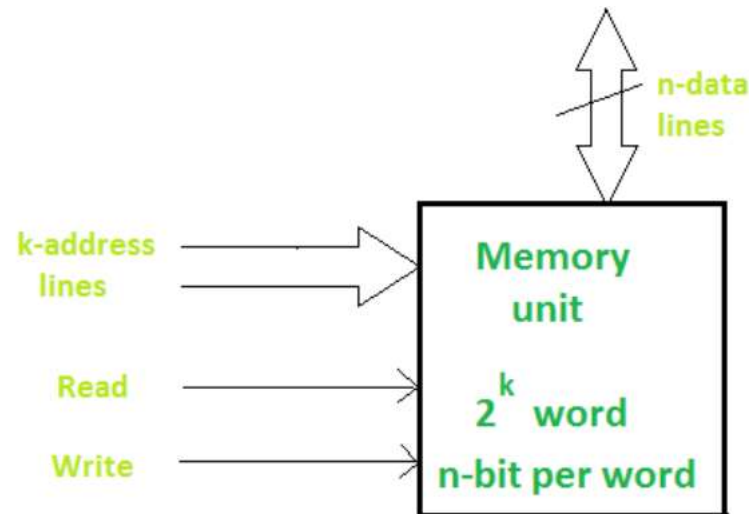
Single-Core CPU: The oldest type, used in the 1970s. It can only handle one task at a time.

Dual-Core CPU: It's faster and better for multitasking, like listening to music while doing homework.

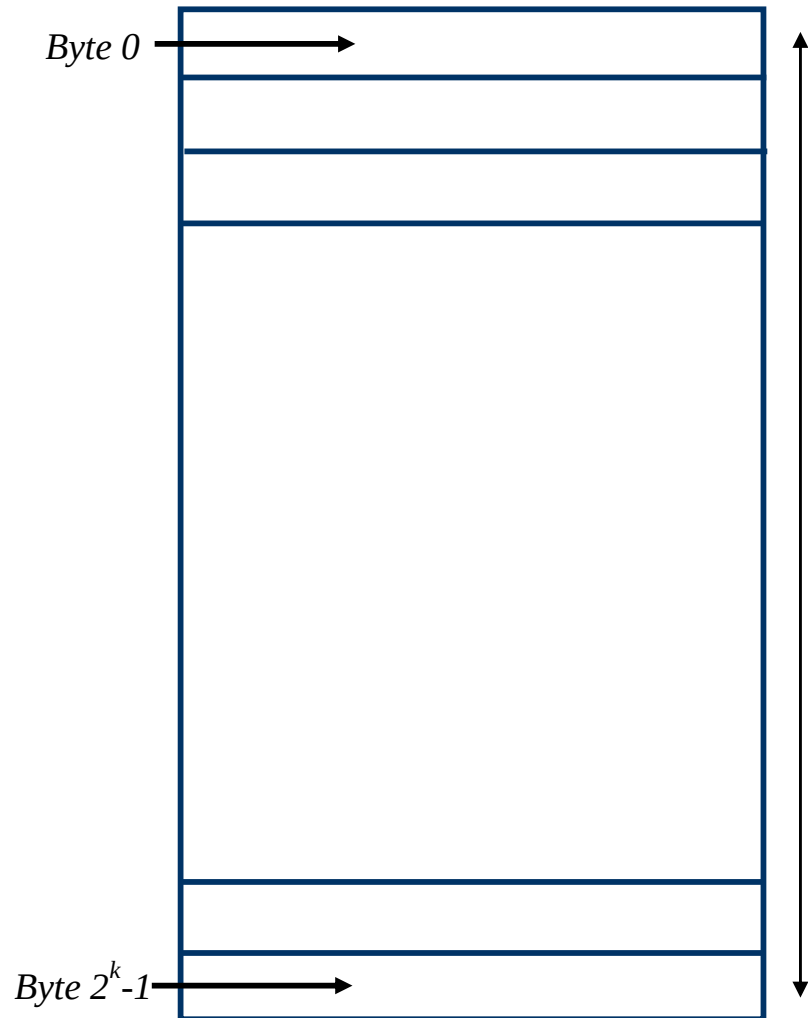
Quad-Core CPU: Has four cores, making it great for heavy tasks like video editing or playing modern games.

Memory

- A memory unit in computer organization is a collection of storage cells, along with associated circuits, used to store binary information (1s and 0s) in the form of words.
- These units are essential for storing data, instructions, and intermediate results during computer operations.
- They can be broadly categorized into main memory (RAM and ROM) and cache memory.
- The data in memory is stored and retrieved by the process called **writing** and **reading**, respectively.



Memory organization (contd..)



- Memory is viewed as a sequence of bytes.
- Address of the first byte is 0
- Address of the last byte is $2^k - 1$, where k is the number of bits used to hold memory address
- E.g. when $k = 16$,
Address of the first byte is 0
Address of the last byte is 65535
- E.g. when $k = 2$,
Address of the first byte is ?
Address of the last byte is ?

Memory organization (contd..)

Word #0	Byte 0
	Byte 1
	Byte 2
	Byte 3
Word #1	Byte 4
Word #?	
	Byte 65532
	Byte 65533
	Byte 65534
	Byte 65535

Consider a memory organization:

16-bit memory addresses

Size of the memory is ?

Word length is 4 bytes

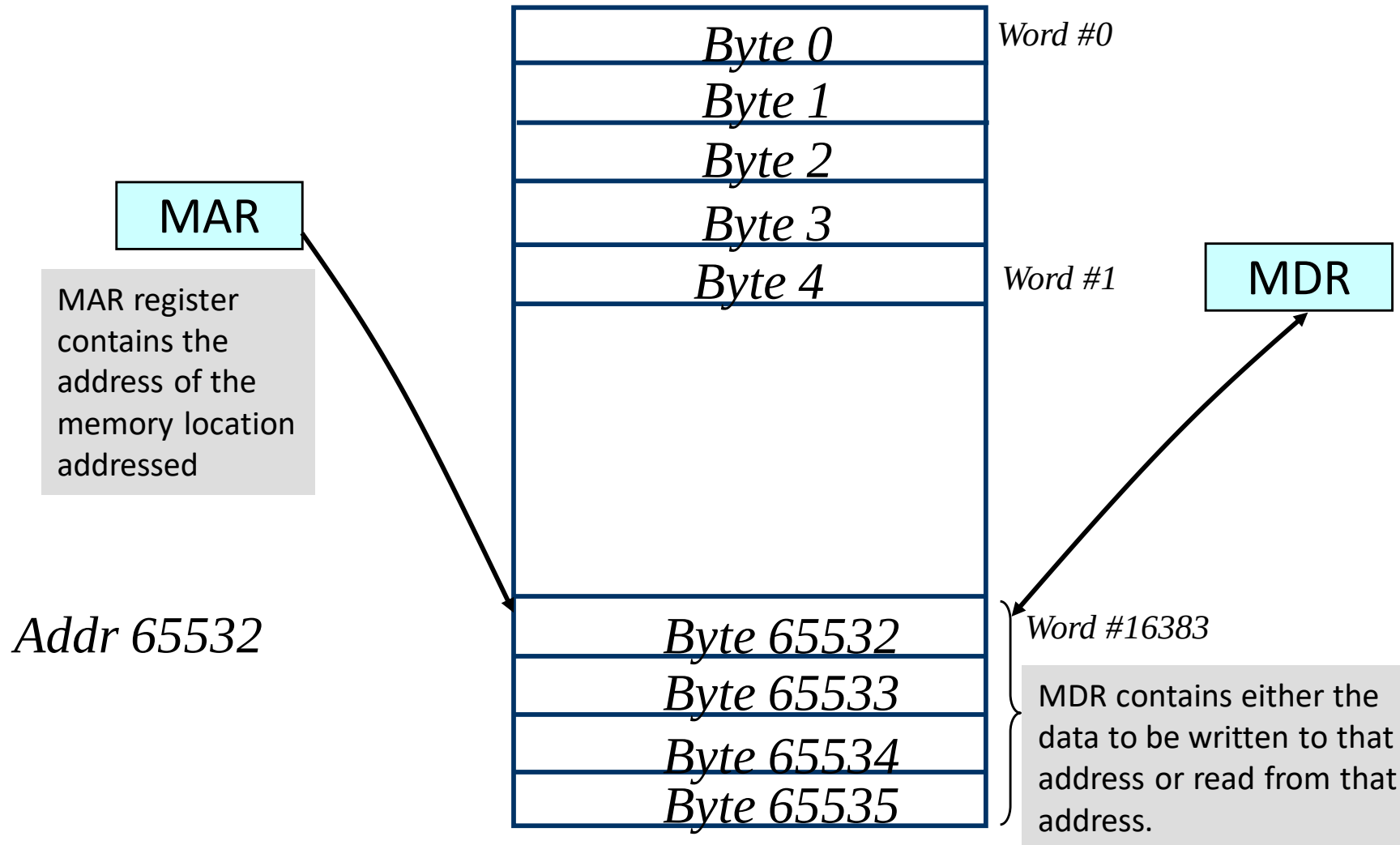
Number of words = $\frac{\text{Memory size(bytes)}}{\text{Word length(bytes)}}$ = ?

Word #0 starts at Byte #0.

Word #1 starts at Byte #4.

Last word (Word #?) starts at Byte#?

Memory organization (contd..)



Memory operations

- **Memory read or load:**

- Place address of the memory location to be read from into MAR.
- Issue a Memory_read command to the memory.
- Data read from the memory is placed into MDR automatically (by control logic).

- **Memory write or store:**

- Place address of the memory location to be written to into MAR.
- Place data to be written into MDR.
- Issue Memory_write command to the memory.
- Data in MDR is written to the memory automatically (by control logic).

Types of Computer Memory

- **Cache Memory:** This temporary storage area, known as a cache, is more readily available to the processor than the computer's main memory source. It is also called CPU memory because it is typically integrated directly into the CPU chip or placed on a separate chip with a bus interconnect with the CPU.
- **RAM:** It is one of the parts of the Main memory, also famously known as Read Write Memory. Random Access memory is present on the motherboard and the computer's data is temporarily stored in RAM.
- **D RAM (Dynamic RAM):** D RAM uses capacitors and transistors and stores the data as a charge on the capacitors. They contain thousands of memory cells.
- **S RAM (Static RAM):** S RAM uses transistors and the circuits of this memory are capable of retaining their state as long as the power is applied. This memory consists of the number of flip flops with each flip flop storing 1 bit.
- **ROM:** ROM full form is Read Only Memory. ROM is a non volatile memory and it is used to store important information which is used to operate the system.

Types of Computer Memory

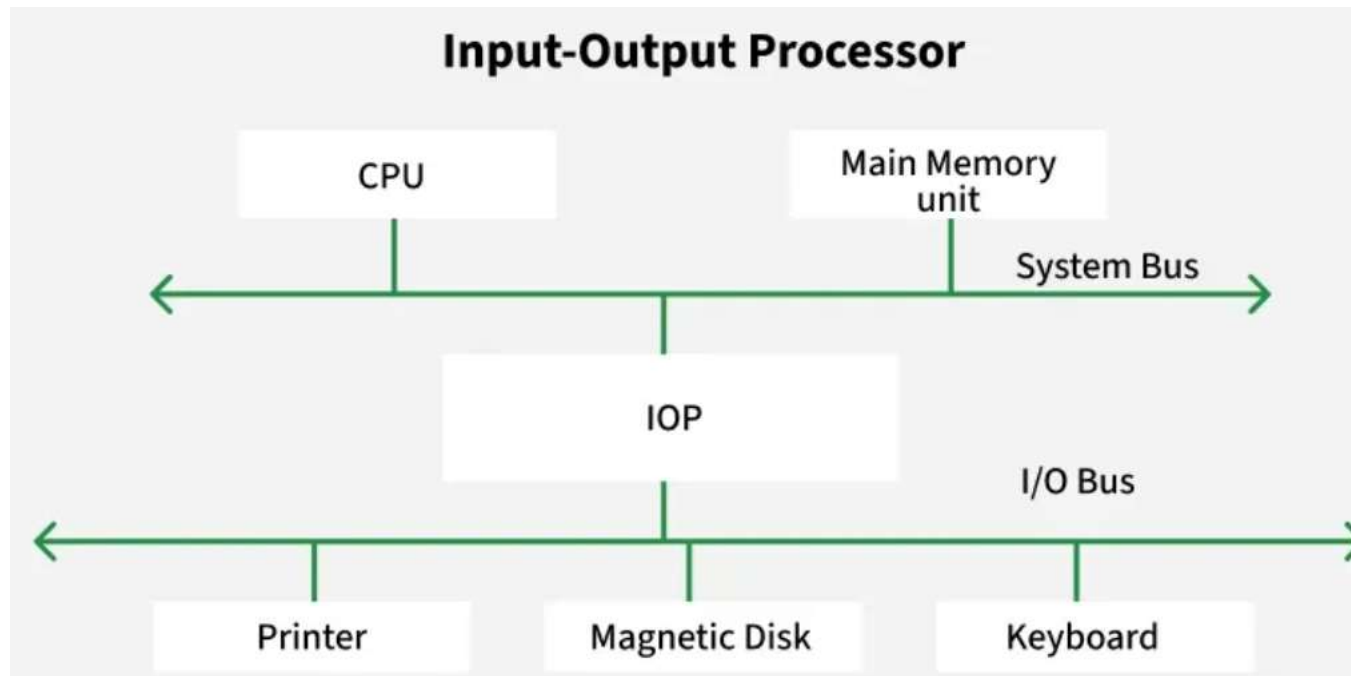
- **MROM(Masked ROM):** Hard-wired devices with a pre-programmed collection of data or instructions were the first ROMs. Masked ROMs are a type of low-cost ROM that works in this way.
- **PROM (Programmable Read Only Memory):** This read-only memory is modifiable once by the user. The user purchases a blank PROM and uses a PROM program to put the required contents into the PROM. Its content can't be erased once written.
- **EPROM (Erasable Programmable Read Only Memory):** EPROM is an extension to PROM where you can erase the content of ROM by exposing it to Ultraviolet rays for nearly 40 minutes.
- **EEPROM (Electrically Erasable Programmable Read Only Memory):** Here the written contents can be erased electrically. You can delete and reprogram EEPROM up to 10,000 times. Erasing and programming take very little time, i.e., nearly 4 -10ms(milliseconds).
- **Virtual memory:** A memory management technique where secondary memory can be used as if it were a part of the main memory. Virtual memory uses hardware and software to enable a computer to compensate for physical memory shortages by temporarily transferring data from RAM to disk storage.

Applications of I/O Processors

- I/O processors can be used in data acquisition systems to handle real-time data transfer and processing.
- They can be used in industrial control systems for precise timing, control signals, and local data processing.
- They can handle multimedia I/O, including real-time data processing for audio, video, and compression.
- They can process network data, including routing, filtering, and encryption.
- They can handle high-speed data transfers, caching, and prefetching for storage systems.

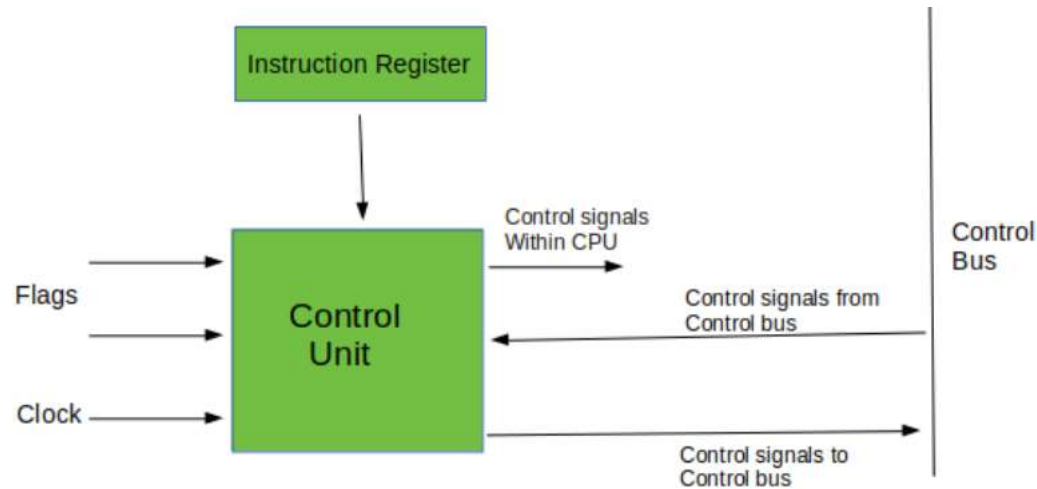
Input-output subsystems

- The I/O (input/output) subsystem of a computer is responsible for communication between the computer's central processing unit (CPU) and the outside world, including peripheral devices and other systems.
- It manages all input and output operations, facilitating data transfer between the CPU, memory, and external devices.



Control unit

- A control unit, or CU, is circuitry within a computer's processor that directs operations.
- It instructs the memory, logic unit, and both output and input devices of the computer on how to respond to the program's instructions.
- CPUs and GPUs are examples of devices that use control units.



Block Diagram of the Control Unit

Types of Control Unit

- There are two types of control units: 1. Hardwired, 2. Micro programmable control unit.

Hardwired Control Unit

- In the Hardwired control unit, the control signals that are important for instruction execution control are generated by specially designed hardware logical circuits, in which we can not modify the signal generation method without a physical change to the circuit structure.
- The operation code of an instruction contains the basic data for control signal generation.

Micro Programmable control unit

- The fundamental difference between these unit structures and the structure of the hardwired control unit is the existence of the control store that is used for storing words containing encoded control signals mandatory for instruction execution.
- In microprogrammed control units, subsequent instruction words are fetched into the instruction register in a normal way.

Instruction set architecture of a CPU—registers

- In a CPU's Instruction Set Architecture (ISA), registers are small, high-speed storage locations within the CPU used to hold data and instructions during processing.
- They are crucial for efficient CPU operation because they allow for quick access to frequently used data, rather than repeatedly fetching it from slower memory.
- The ISA is not concerned with the implementation-specific details of a computer. It is only concerned with the set or collection of basic operations that the computer must support.
- For example, the Advanced Micro Devices (AMD) Athlon and the Core 2 Duo processors have entirely different implementations, but they support more or less the same set of basic operations as defined in the x86 Instruction Set.

Instruction set architecture of a CPU—registers

Based on the type of operations they perform MIPS Instructions are classified into 3 types:

➤ **Arithmetic/Logic Instructions:**

➤ These Instructions perform various Arithmetic & Logical operations on one or more operands.

➤ **Data Transfer Instructions:**

➤ These instructions are responsible for the transfer of instructions from memory to the processor registers and vice versa.

➤ **Branch and Jump Instructions:**

➤ These instructions are responsible for breaking the sequential flow of instructions and jumping to instructions at various other locations. This is necessary for the implementation of functions and conditional statements.

Different Types Of CPU Registers

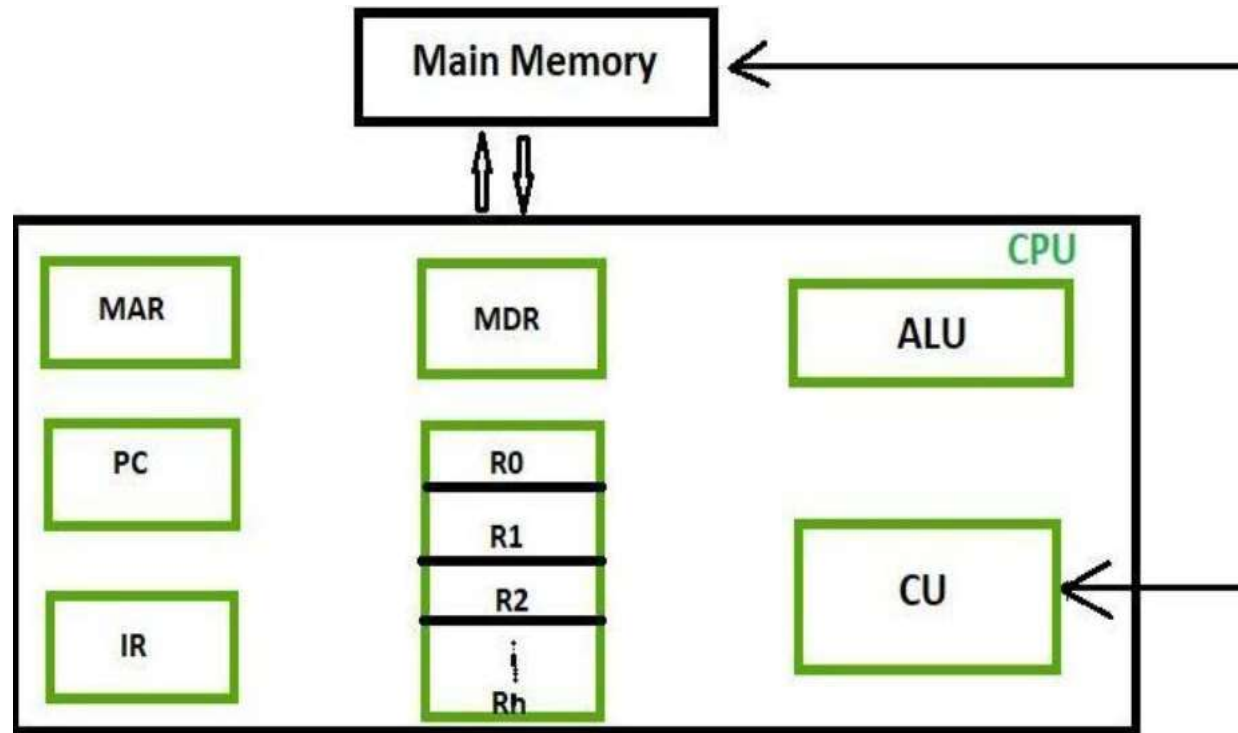
There are several types of registers available in the CPU for different purposes let's discuss each one by one:

- **Accumulator** : This is the most frequently used register used to store data taken from memory. It is indifferent numbers in different microprocessors.
- **Memory Address Registers (MAR)** : It holds the address of the location to be accessed from memory. MAR and MDR (Memory Data Register) together facilitate the communication of the CPU and the main memory.
- **Memory Data Registers (MDR)** : It contains data to be written into or to be read out from the addressed location.
- **General Purpose Registers** : These are numbered as R0, R1, R2....Rn-1, and used to store temporary data during any ongoing operation. Its content can be accessed by assembly programming. Modern CPU architectures tend to use more GPR so that register-to-register addressing can be used more, which is comparatively faster than other addressing modes .
- **Program Counter (PC)** : Program Counter (PC) is used to keep the track of the execution of the program. It contains the memory address of the next instruction to be fetched. PC points to the address of the next instruction to be fetched from the main memory when the previous instruction has been successfully completed.

Different Types of CPU Registers

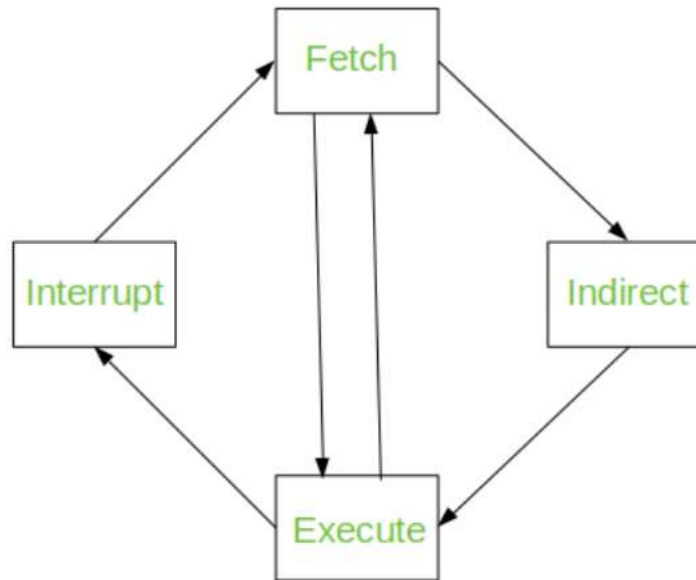
- **Instruction Register (IR):** The IR holds the instruction which is just about to be executed. The instruction from the PC is fetched and stored in IR. As soon as the instruction is placed in IR, the CPU starts executing the instruction, and the PC points to the next instruction to be executed.
- **Stack Pointer (SP):** The stack PC's pointer points to the top of the stack, which is a part of the memory used to store function calls and other operations.
- **Flag Register:** A flag register, also known as a status register or condition code register, is a special type of register in a computer's central processing unit (CPU) used to indicate the status of the CPU or the outcome of various operations such as Zero Flag, Carry flag, Sign Flag, Overflow Flag, Parity Flag, Auxiliary Carry Flag, and Interrupt Enable Flag.
- **Condition code register (CCR):** Condition code registers contain different flags that indicate the status of any operation. For instance, let's suppose an operation caused the creation of a negative result or zero, then these flags are set high accordingly.

Different Types of CPU Registers



Instruction execution cycle

- The instruction execution cycle, also known as the fetch-decode-execute cycle, is the basic process a CPU uses to execute instructions in a program.
- It involves fetching an instruction from memory, decoding it to understand the operation, and then executing that operation.
- This cycle repeats continuously for each instruction in a program.



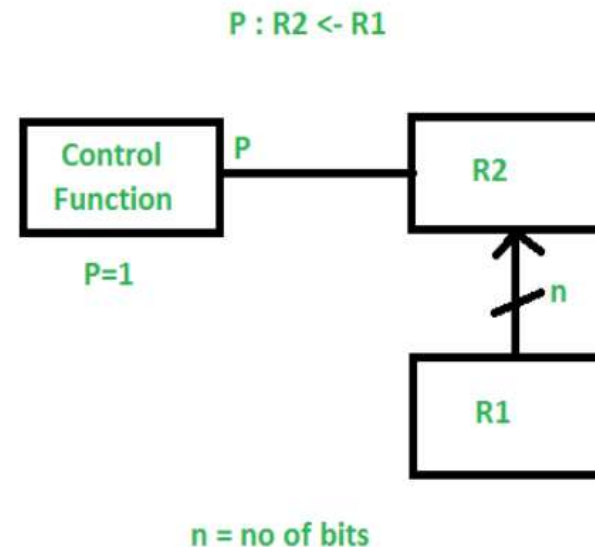
The Instruction Cycle

Instruction execution cycle

- **Fetch:** In the fetch cycle, the CPU retrieves the instruction from memory. The instruction is typically stored at the address specified by the program counter (PC). The PC is then incremented to point to the next instruction in memory.
- **Decode:** In the decode cycle, the CPU interprets the instruction and determines what operation needs to be performed. This involves identifying the opcode and any operands that are needed to execute the instruction.
- **Execute:** In the execute cycle, the CPU performs the operation specified by the instruction. This may involve reading or writing data from or to memory, performing arithmetic or logic operations on data, or manipulating the control flow of the program.
- **Fetch operands:** In some CPUs, the operands needed for an instruction are fetched during a separate cycle before the execute cycle. This is called the fetch operands cycle.
- **Store results:** In some CPUs, the results of an instruction are stored during a separate cycle after the execute cycle. This is called the store results cycle.
- **Interrupt handling:** In some CPUs, interrupt handling may occur during any cycle of the instruction cycle. An interrupt is a signal that the CPU receives from an external device or software that requires immediate attention.

RTL interpretation of instructions

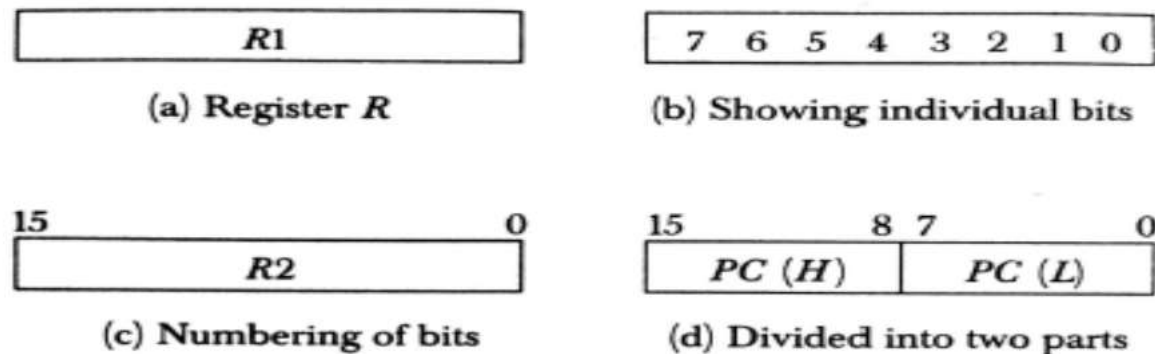
- Register Transfer Language (RTL) is a symbolic notation used to describe the movement and manipulation of data between registers within a digital system.
- It acts as an intermediate representation between high-level descriptions and hardware implementation, allowing designers to define and verify the functionality of a digital circuit before physical construction.
- RTL focuses on the flow of data between registers, rather than the specific gates and transistors, offering a higher level of abstraction.



Registers

- Computer registers are designated by upper case letters (and optionally followed by digits or letters) to denote the function of the register.
- For example, the register that holds an address for the memory unit is usually called a memory address register and is designated by the name MAR.
- Other designations for registers are PC (for program counter), IR (for instruction register, and R1 (for processor register).
- The individual flip-flops in an n-bit register are numbered in sequence from 0 through n-1, starting from 0 in the rightmost position and increasing the numbers toward the left.

Figure 4-1 Block diagram of register.



Addressing modes

- Addressing modes are the techniques used by a CPU to determine the location of an operand (the data being operated on) within an instruction.
- They define how the CPU interprets the address field of an instruction to locate the data it needs to process.
- Different addressing modes offer varying levels of flexibility and efficiency in accessing data in memory or registers.

Importance of Addressing Modes

- **Flexibility:** Allows programmers to choose the most efficient way to access data based on program requirements.
- **Efficiency:** Optimizes memory usage and execution speed by providing different methods of accessing operands.
- **Complexity Reduction:** Simplifies instruction formats and enables more concise, powerful code.
- **Enhanced Performance:** Improves CPU performance by reducing the number of memory accesses required for certain operations.

Addressing Modes Types

- The addressing modes refer to how someone can address any given memory location.
- Implied Mode
- Immediate Mode
- Register Mode
- Register Indirect Mode
- Autodecrement Mode
- Autoincrement Mode
- Direct Address Mode
- Indirect Address Mode
- Indexed Addressing Mode

Addressing Modes Types

- The addressing modes refer to how someone can address any given memory location.
- Implied/ Implicit Addressing Mode
- Immediate Addressing Mode
- Register Direct Addressing Mode
- Register Indirect Addressing Mode
- Auto-Increment Addressing Mode
- Auto-Decrement Addressing Mode
- Direct Addressing Mode
- Indirect Addressing Mode
- Displacement Addressing Mode
- Relative Addressing Mode
- Indexed Addressing Mode
- Base Register Addressing Mode
- Stack Addressing Mode

Addressing Modes Types

➤ **Implied/ Implicit Addressing Mode**

- The operands are implicitly specified in the instruction's definition.
- Consider the example, the instruction “complement accumulator” is an implied-mode instruction as the operand in the accumulator register is implied in the instruction definition.
- All register reference the instructions that use an accumulator are implied-mode instructions.
- Zero-address instructions in a stack-organized computer are also implied-mode instructions because the operands are implied to be on top of the stack.

Addressing Modes Types

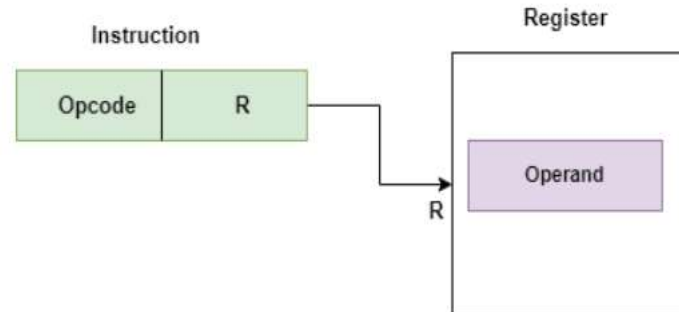
➤ Immediate Addressing Mode

- The operand is defined in the instruction itself. This mode instruction has an operand field instead of an address field. The operand field contains the actual operand used with the specified operation in the instruction. The immediate-mode instructions help initialize registers to a constant value.



➤ Register Direct Addressing Mode

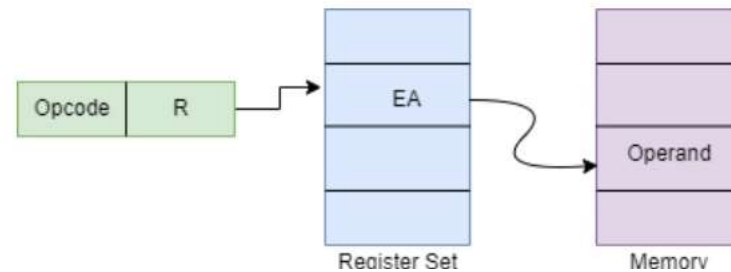
- The operands that reside within the CPU are stored in the registers. The specific register is selected from a register field in the instruction. No reference to the memory is required to fetch the operand. The only difference between the Direct addressing mode and the register direct addressing mode is that the instruction address field refers to a CPU register instead of the main memory.



Addressing Modes Types

➤ Register Indirect Addressing Mode

- The instruction defines a register in the CPU that stores the effective address of the operand in memory. Only one reference to the memory is required to fetch the operand. The specified register contains the address of the operand instead of the operand. The only difference between the Indirect addressing mode and the register indirect addressing mode is that the instruction address field refers to a CPU register.

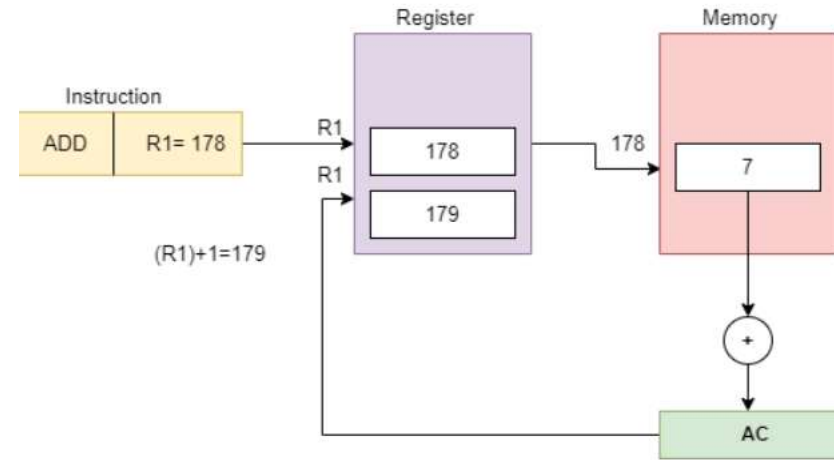


➤ Auto-Increment Addressing Mode

- This mode is a special Register Indirect Addressing Mode case. The register is incremented/decremented after or before its value is utilized.
- $EA = \text{content of the register}$.

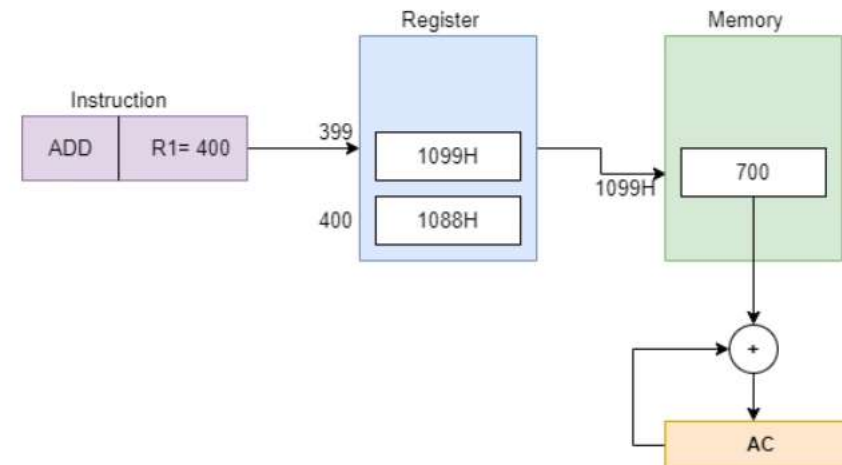
Addressing Modes Types

➤ Auto-Increment Addressing Mode



➤ Auto-Decrement Addressing Mode

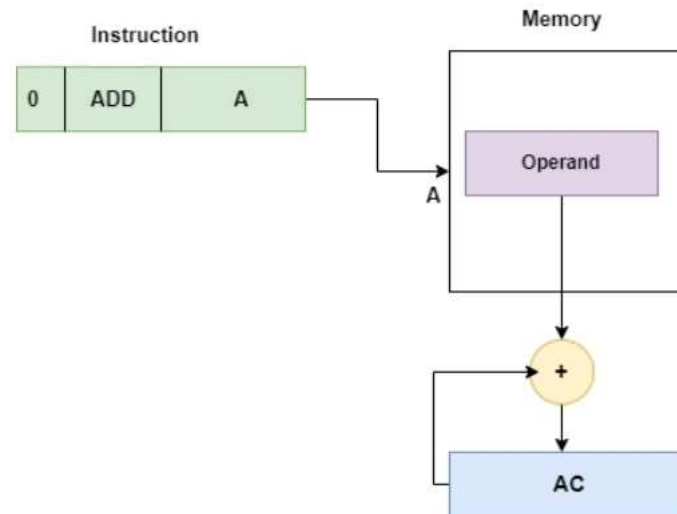
- This mode is also a special Register Indirect Addressing Mode case.
- $EA = \text{content of the register} - \text{step size}$.
- The content of the register is decremented by step size 'd' after accessing the operand, where the step size 'd' depends on the size of the accessed operand. Only one reference memory is required to fetch the operand.



Addressing Modes Types

➤ Direct Addressing Mode

- The effective address of the operand resides in the address field of the instruction. The operand resides in the memory, and the address field of the instruction gives its address. Only one reference to the memory is required to fetch the operand, and no additional calculations need to be done to find the effective address of the operand. It is also known as absolute addressing mode.



Addressing Modes Types

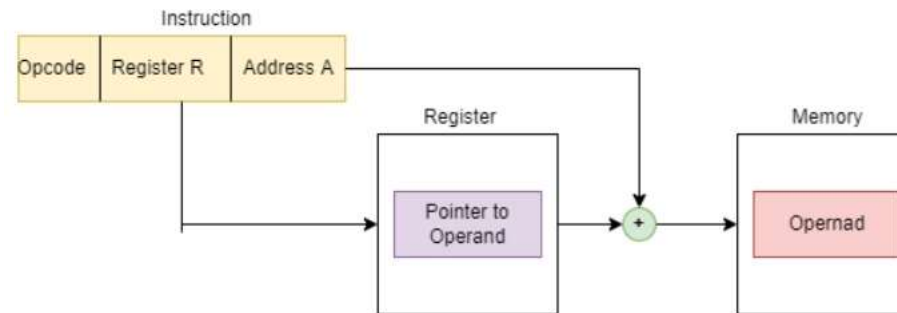
➤ Indirect Addressing Mode

- The address field of the instruction gives the address of the memory location that contains the effective address of the operand. Two references to the memory are required to fetch the operand: The control fetches the instruction from memory and uses its address part to access the memory that stores the effective address. This addressing mode slows down the execution as it requires multiple memory lookups to find the operand.



➤ Displacement Addressing Mode

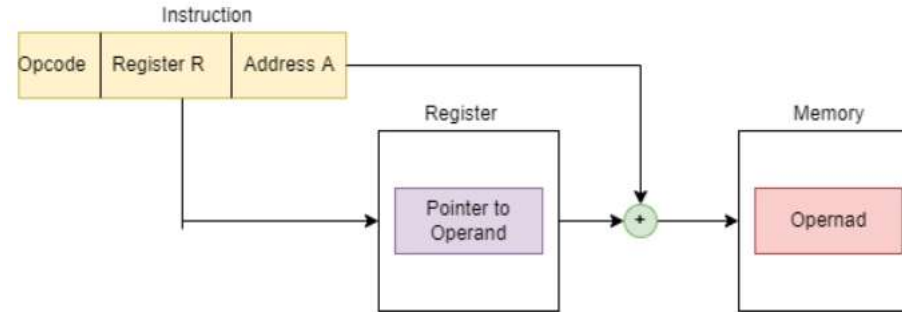
- The indexed register content is added to the instruction's address part to obtain the effective address of the operand.



Addressing Modes Types

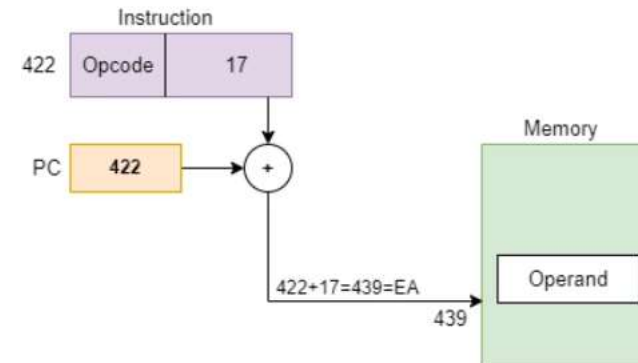
➤ Displacement Addressing Mode

- The indexed register content is added to the instruction's address part to obtain the effective address of the operand.



➤ Relative Addressing Mode

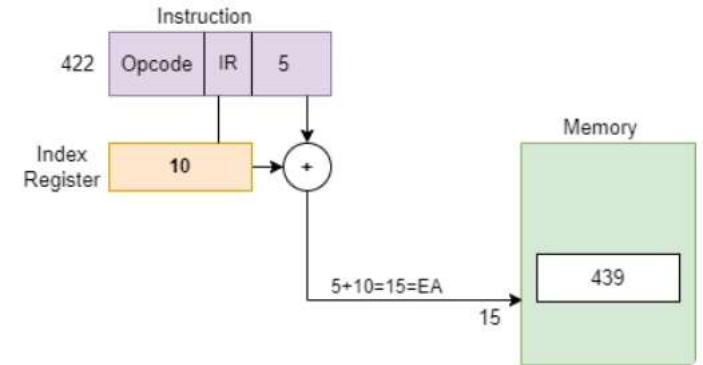
- This mode is another version of the displacement address mode. The program counter's content is added to the instruction's address part to obtain the effective address.
- $EA = A + (PC)$
- Here, EA: Effective address, PC: program counter.



Addressing Modes Types

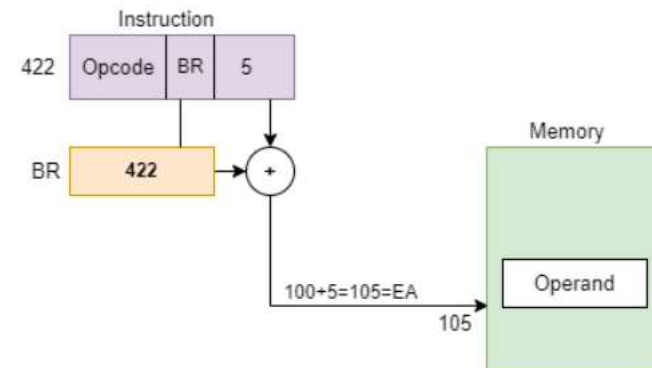
➤ Indexed Addressing Mode

- The index register's content is added to the instruction's address to obtain the effective address.
- $EA = \text{content of index register} + \text{Instruction address part}$.



➤ Base Register Addressing Mode

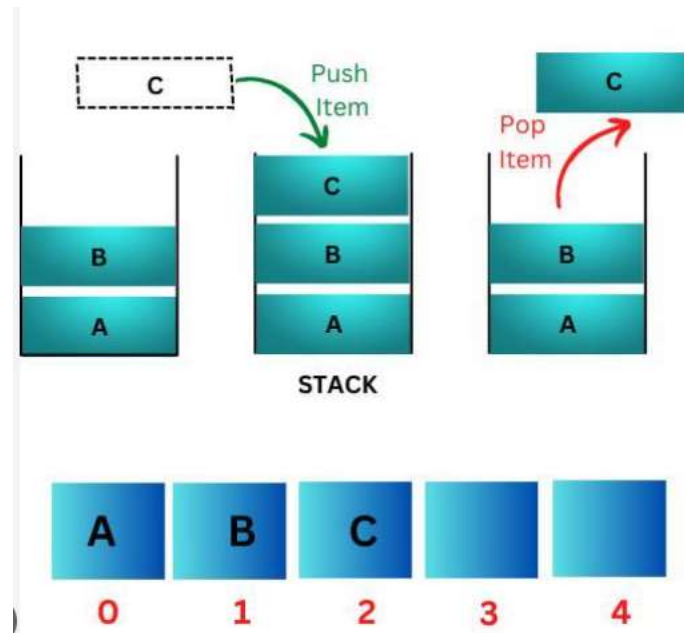
- This mode is another version of the displacement address mode. To obtain the effective address, the base register's content is added to the instruction's address.
- $EA = A + (R)$
- A: Displacement, R: Pointer to the base address.



Addressing Modes Types

➤ Stack Addressing Mode

- The operand resides at the top of the stack. Consider an example, ADD: This instruction will POP two items from the stack, add them, and at last, will PUSH the result to the memory references of Addressing Modes.
- It helps in reducing the number of bits in the instruction's addressing field.
- It facilitates pointers, indexing of data, and counters for loop controls.



Addressing Modes Types

➤ **Advantages of Addressing Modes**

- They improve performance by efficiently utilizing the CPU cache and reducing the memory read latency.
- Addressing Modes are used for implementing complex data structures in memory as they provide mechanisms such as indexing.
- Program sizes can be reduced drastically as the code can be compacted which allows faster execution of instructions.
- Addressing modes give you the flexibility to use different ways of specifying the address of operands in your instructions.

➤ **Disadvantages of Addressing Modes**

- **Complexity:** Particularly for inexperienced programmers, some addressing modes might be challenging to comprehend and use. Due to its intricacy, code may include errors or be inefficient.
- **Limited Flexibility:** Some addressing modes may limit the kind of operand operations that can be carried out. For instance, Immediate Mode restricts dynamic calculations to preset values only.
- **Increased Memory Access:** Because Direct and Indirect modes need more memory accesses to retrieve operands, program performance may be slowed down.
- **Register Dependency:** The availability of registers is very important for register modes. This can cause conflicts and inefficient resource utilization in systems with a finite number of registers.
- **Code Size:** Certain addressing modes could provide longer and more complicated instructions, increasing the code size.

Types of instructions

- In computer architecture, instructions can be broadly classified into several types based on their function.
- These include data transfer instructions, data manipulation instructions (arithmetic and logical), and program control instructions.
- Additionally, instructions can be categorized by their format (e.g., zero, one, two, or three address instructions).

Computer instructions must be capable of performing **4 types** of operations.

Data transfer/movement between memory and processor registers.

E.g., memory read, memory write

Arithmetic and logic operations:

E.g., addition, subtraction, comparison between two numbers.

Program sequencing and flow of control:

Branch instructions

Input/output transfers to transfer data to and from the real world.

Types of instructions

- **Data transfer/movement between memory and processor registers**
- These instructions are responsible for moving data between different locations within the computer system.
- Examples include MOV (move data), LOAD (load data from memory), and STORE (store data to memory).
- They facilitate the movement of data between registers, memory, and input/output devices.

Mode	Assembly	Register Transfer
Direct address	LD <i>ADR</i>	$AC \leftarrow M[ADR]$
Indirect address	LD <i>@ADR</i>	$AC \leftarrow M[M[ADR]]$
Relative address	LD <i>\$ADR</i>	$AC \leftarrow M[PC+ADR]$
Immediate operand	LD <i>#NBR</i>	$AC \leftarrow NBR$
Index addressing	LD <i>ADR(X)</i>	$AC \leftarrow M[ADR+XR]$
Register	LD <i>R1</i>	$AC \leftarrow R1$
Register indirect	LD <i>(R1)</i>	$AC \leftarrow M[R1]$
Autoincrement	LD <i>(R1)+</i>	$AC \leftarrow M[R1], R1 \leftarrow R1+1$

Types of instructions

➤ Data Manipulation Instructions:

- These instructions perform operations on data, such as arithmetic and logical computations.
- Arithmetic instructions: include operations like addition (ADD), subtraction (SUB), multiplication (MUL), and division (DIV).
- Logical instructions: perform bitwise operations like AND, OR, XOR, and NOT.

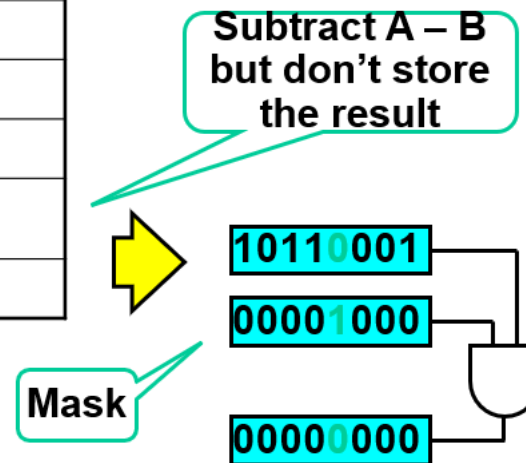
Name	Mnemonic
Clear	CLR
Complement	COM
AND	AND
OR	OR
Exclusive-OR	XOR
Clear carry	CLRC
Set carry	SETC
Complement carry	COMC
Enable interrupt	EI
Disable interrupt	DI

Name	Mnemonic
Logical shift right	SHR
Logical shift left	SHL
Arithmetic shift right	SHRA
Arithmetic shift left	SHLA
Rotate right	ROR
Rotate left	ROL
Rotate right through carry	RORC
Rotate left through carry	ROLC

Types of instructions

- **Program Control Instructions:**
- These instructions manage the flow of execution within a program.
- They include instructions like JUMP (unconditional transfer of control), BRANCH (conditional transfer of control), and CALL and RETURN (for function calls and returns).
- They are crucial for implementing loops, conditional statements, and function calls.

Name	Mnemonic
Branch	BR
Jump	JMP
Skip	SKP
Call	CALL
Return	RET
Compare (Subtract)	CMP
Test (AND)	TST



Types of instructions

➤ Instruction Formats:

Zero-address instructions:

These instructions operate on a stack, where operands are implicitly available on the top of the stack.

One-address instructions:

These instructions typically use an accumulator register as one of the operands, with the other operand being specified in memory.

Two-address instructions:

These instructions specify two operands, one source and one destination.

Three-address instructions:

These instructions specify three operands: two sources and one destination.

Mnemonic	Branch Condition	Tested Condition
BZ	Branch if zero	$Z = 1$
BNZ	Branch if not zero	$Z = 0$
BC	Branch if carry	$C = 1$
BNC	Branch if no carry	$C = 0$
BP	Branch if plus	$S = 0$
BM	Branch if minus	$S = 1$
BV	Branch if overflow	$V = 1$
BNV	Branch if no overflow	$V = 0$

Instruction set

- An instruction set is a collection of commands that a computer processor can understand and execute, essentially telling the CPU what operations to perform.
- It acts as the interface between software and hardware, allowing programmers to control the CPU through machine language.

Instruction Set Architecture (ISA)

- ISA is the **interface** between hardware and software.
- ISA is what is visible to the programmer (and ISA might be different for O.S. and applications)

ISA consists of:

- instructions (operations and how they are encoded)
- information units (size, how they are addressed etc.)
- registers (or more generally, processor state)
- input-output control

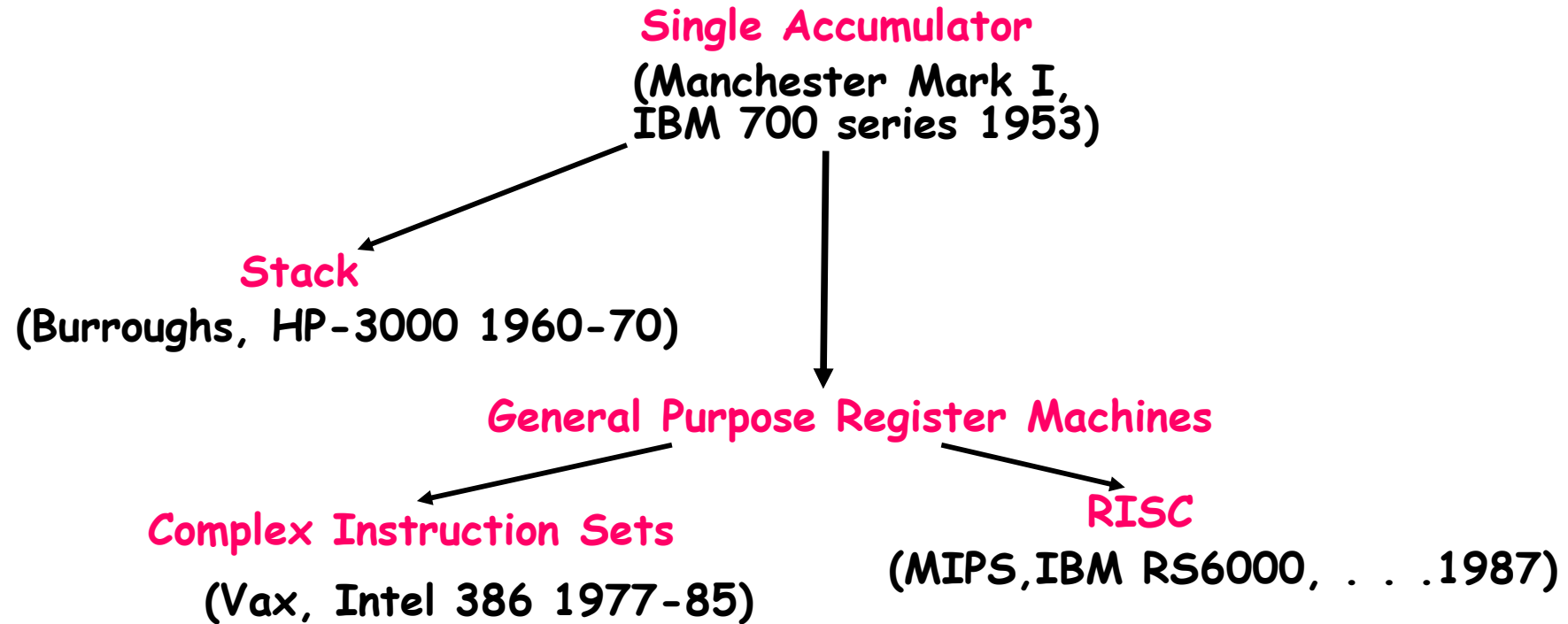
Register Transfer Notation

- Identify a location by a symbolic name standing for its hardware binary address (LOC, R0,...)
- Contents of a location are denoted by placing square brackets around the name of the location ($R1 \leftarrow [LOC]$, $R3 \leftarrow [R1] + [R2]$)
- Register Transfer Notation (RTN)

Assembly Language Notation

- Represent machine instructions and programs.
- Move LOC, $R1 = R1 \leftarrow [LOC]$
- Add R1, R2, $R3 = R3 \leftarrow [R1] + [R2]$

Evolution of Instruction Sets



Comparison of Architectures

Consider the operation: $C = A + B$

Stack	Accumulator	Register-Memory	Register-Register
Push A	Load A	Load R1, A	Load R1, A
Push B	Add B	Add R1, B	Load R2, B
Add	Store C	Store C, R1	Add R3, R1, R2
Pop C			Store C, R3

Instruction Formats

- Three-Address Instructions

- ADD R1, R2, R3 $R1 \leftarrow R2 + R3$

- Two-Address Instructions

- ADD R1, R2 $R1 \leftarrow R1 + R2$

- One-Address Instructions

- ADD M $AC \leftarrow AC + M[AR]$

- Zero-Address Instructions

- ADD $TOS \leftarrow TOS + (TOS - 1)$

- RISC Instructions

- Lots of registers. Memory is restricted to Load & Store



Instruction Formats

Example: Evaluate $(A+B) * (C+D)$

- Zero-Address

1. PUSH	A	; TOS $\leftarrow$ A
2. PUSH	B	; TOS $\leftarrow$ B
3. ADD		; TOS $\leftarrow$ (A + B)
4. PUSH	C	; TOS $\leftarrow$ C
5. PUSH	D	; TOS $\leftarrow$ D
6. ADD		; TOS $\leftarrow$ (C + D)
7. MUL		; TOS $\leftarrow$ (C+D)*(A+B)
8. POP	X	; M[X] $\leftarrow$ TOS



Instruction Formats

Example: Evaluate $(A+B) * (C+D)$

- One-Address

1. LOAD A

; $AC \leftarrow M[A]$

2. ADD B

; $AC \leftarrow AC + M[B]$

3. STORE T

; $M[T] \leftarrow AC$

4. LOAD C

; $AC \leftarrow M[C]$

5. ADD D

; $AC \leftarrow AC + M[D]$

6. MUL T

; $AC \leftarrow AC * M[T]$

7. STORE X

; $M[X] \leftarrow AC$



Instruction Formats

Example: Evaluate $(A+B) * (C+D)$

- Two-Address

- | | | |
|--------|--------|-----------------------------|
| 1. MOV | R1, A | ; $R1 \leftarrow M[A]$ |
| 2. ADD | R1, B | ; $R1 \leftarrow R1 + M[B]$ |
| 3. MOV | R2, C | ; $R2 \leftarrow M[C]$ |
| 4. ADD | R2, D | ; $R2 \leftarrow R2 + M[D]$ |
| 5. MUL | R1, R2 | ; $R1 \leftarrow R1 * R2$ |
| 6. MOV | X, R1 | ; $M[X] \leftarrow R1$ |



Instruction Formats

Example: Evaluate $(A+B) * (C+D)$

- Three-Address

1. ADD R1, A, B

; $R1 \leftarrow M[A] + M[B]$

2. ADD R2, C, D

; $R2 \leftarrow M[C] + M[D]$

3. MUL X, R1, R2

; $M[X] \leftarrow R1 * R2$



Instruction Formats

Example: Evaluate $(A+B) * (C+D)$

- RISC

- | | | |
|----------|------------|---------------------------|
| 1. LOAD | R1, A | ; $R1 \leftarrow M[A]$ |
| 2. LOAD | R2, B | ; $R2 \leftarrow M[B]$ |
| 3. LOAD | R3, C | ; $R3 \leftarrow M[C]$ |
| 4. LOAD | R4, D | ; $R4 \leftarrow M[D]$ |
| 5. ADD | R1, R1, R2 | ; $R1 \leftarrow R1 + R2$ |
| 6. ADD | R3, R3, R4 | ; $R3 \leftarrow R3 + R4$ |
| 7. MUL | R1, R1, R3 | ; $R1 \leftarrow R1 * R3$ |
| 8. STORE | X, R1 | ; $M[X] \leftarrow R1$ |



CISC

- The primary goal of CISC architecture is to complete a task in as few lines of assembly as possible.
- This is achieved by building processor hardware that is capable of understanding and executing a series of operations.
- For this particular task, a CISC processor would come prepared with a specific instruction (we'll call it "MULT").
- When executed, this instruction loads the two values into separate registers, multiplies the operands in the execution unit, and then stores the product in the appropriate register.
- Thus, the entire task of multiplying two numbers can be completed with one instruction.

Characteristics of CISC

- Complex instruction, hence complex instruction decoding.
- Instructions are larger than one-word size.
- Instruction may take more than a single clock cycle to get executed.
- Less number of general-purpose registers as operations get performed in memory itself.
- Complex Addressing Modes.
- More Data types.

RISC

➤ The main idea behind this is to simplify hardware by using an instruction set composed of a few basic steps for loading, evaluating, and storing operations just like a load command will load data, a store command will store the data.

Characteristics of RISC

- Simpler instruction, hence simple instruction decoding.
- Instruction comes undersize of one word.
- Instruction takes a single clock cycle to get executed.
- More general-purpose registers.
- Simple Addressing Modes.
- Fewer Data types.
- A pipeline can be achieved.

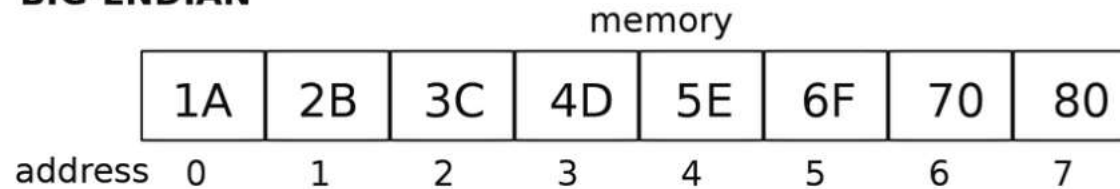
Difference between RISC and CISC

S.No.	RISC	CISC
1.	It stands for Reduced Instruction Set Computer.	It stands for Complex Instruction Set Computer.
2.	It is a microprocessor architecture that uses small instruction set of uniform length.	This offers hundreds of instructions of different sizes to the users.
3.	These simple instructions are executed in one clock cycle.	This architecture has a set of special purpose circuits which help execute the instructions at a high speed.
4.	These chips are relatively simple to design.	These chips are complex to design.
5.	They are inexpensive.	They are relatively expensive.
6.	Examples of RISC chips include SPARC, POWER PC.	Examples of CISC include Intel architecture, AMD.
7.	It has less number of instructions.	It has more number of instructions.
8.	It has fixed-length encodings for instructions.	It has variable-length encodings of instructions.

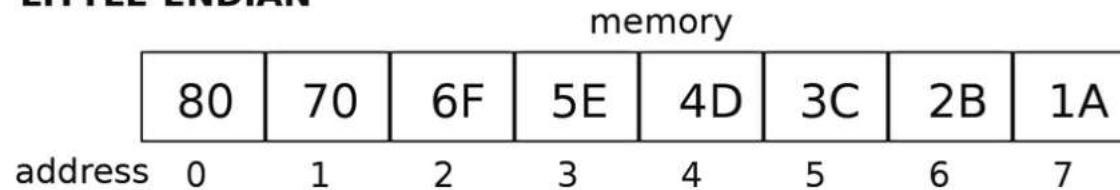
Big-endian & Little-endian representation

- Big-endian and little-endian are two ways to represent the order of bytes in computer memory when storing multi-byte data.
- In big-endian, the most significant byte (MSB) is stored at the lowest memory address, while in little-endian, the least significant byte (LSB) is stored at the lowest memory address.

BIG-ENDIAN



LITTLE-ENDIAN



Memory Location, Addresses, and Operation

- Memory consists of many millions of storage cells, each of which can store 1 bit.
- Data is usually accessed in n -bit groups. n is called word length.

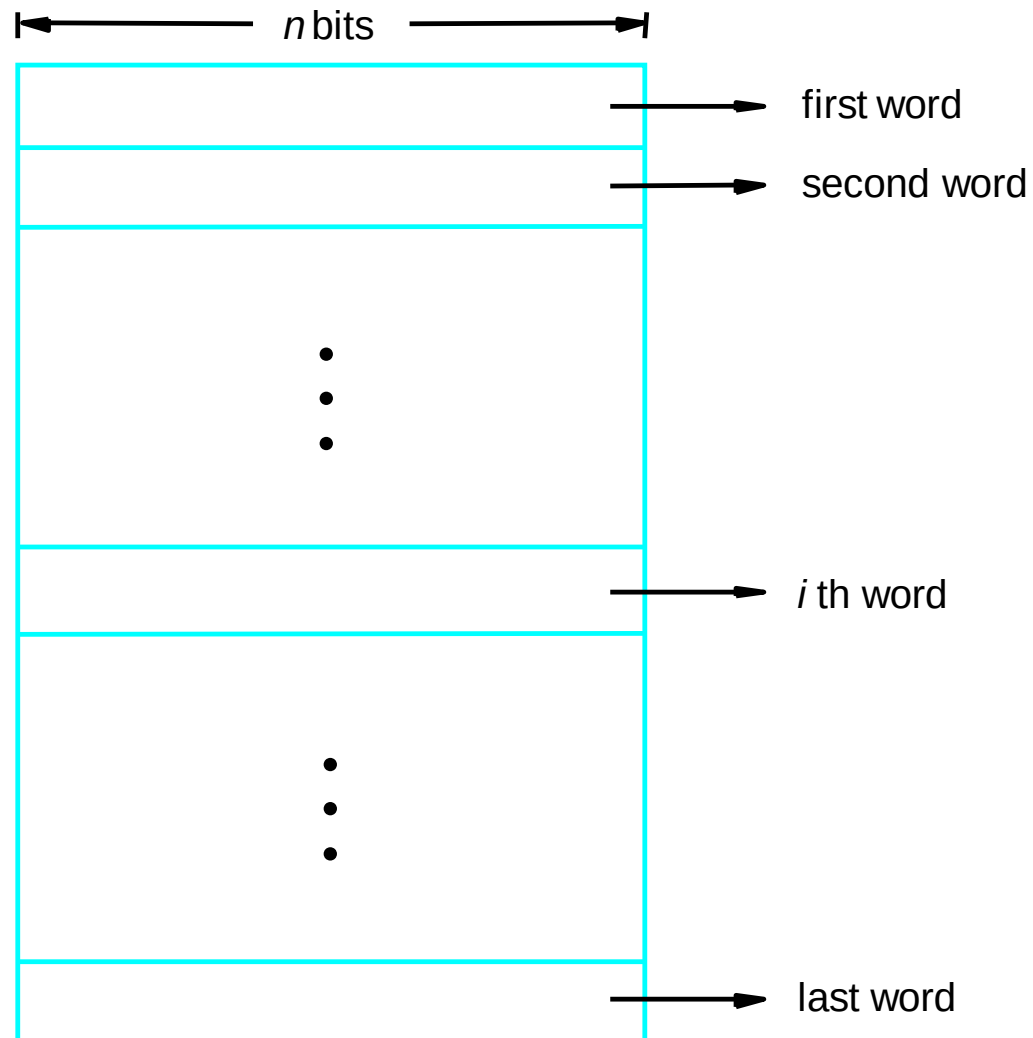
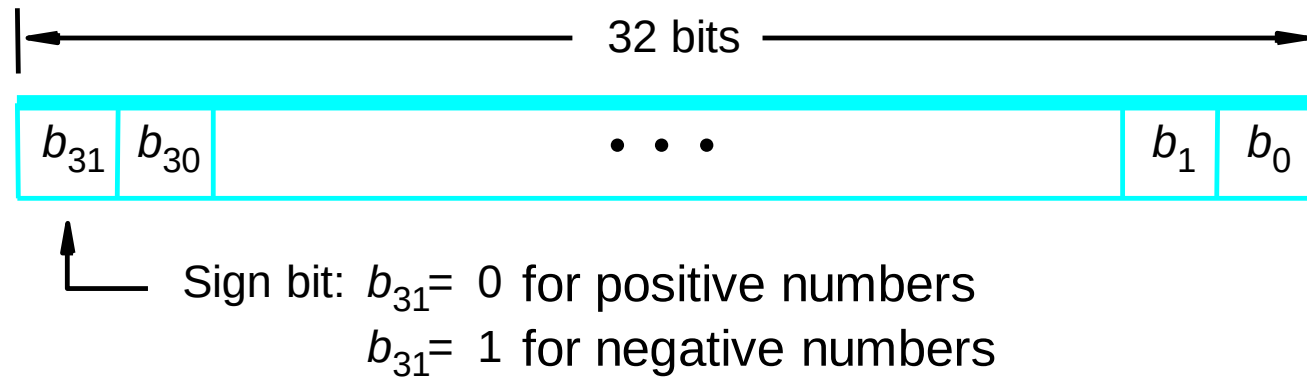


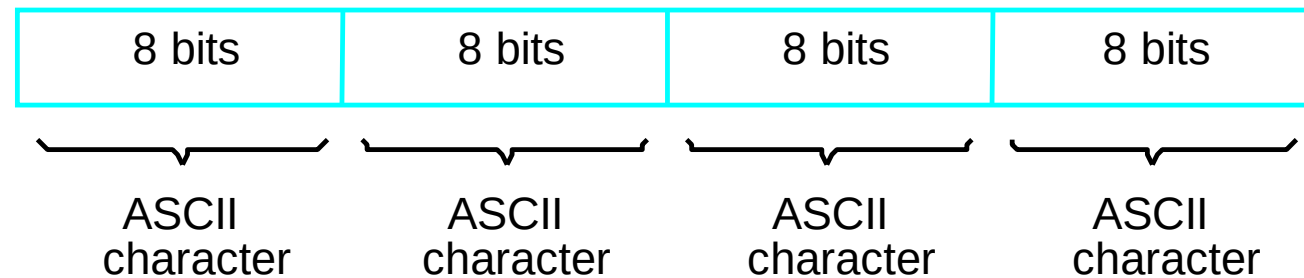
Figure 2.5. Memory words.

Memory Location, Addresses, and Operation

- 32-bit word length example



(a) A signed integer



(b) Four characters

Memory Location, Addresses, and Operation

- To retrieve information from memory, either for one word or one byte (8-bit), addresses for each location are needed.
- A k -bit address memory has 2^k memory locations, namely $0 - 2^k - 1$, called memory space.
- 24-bit memory: $2^{24} = 16,777,216 = 16\text{M}$ ($1\text{M} = 2^{20}$)
- 32-bit memory: $2^{32} = 4\text{G}$ ($1\text{G} = 2^{30}$)
- $1\text{K(kilo)} = 2^{10}$
- $1\text{T(tera)} = 2^{40}$

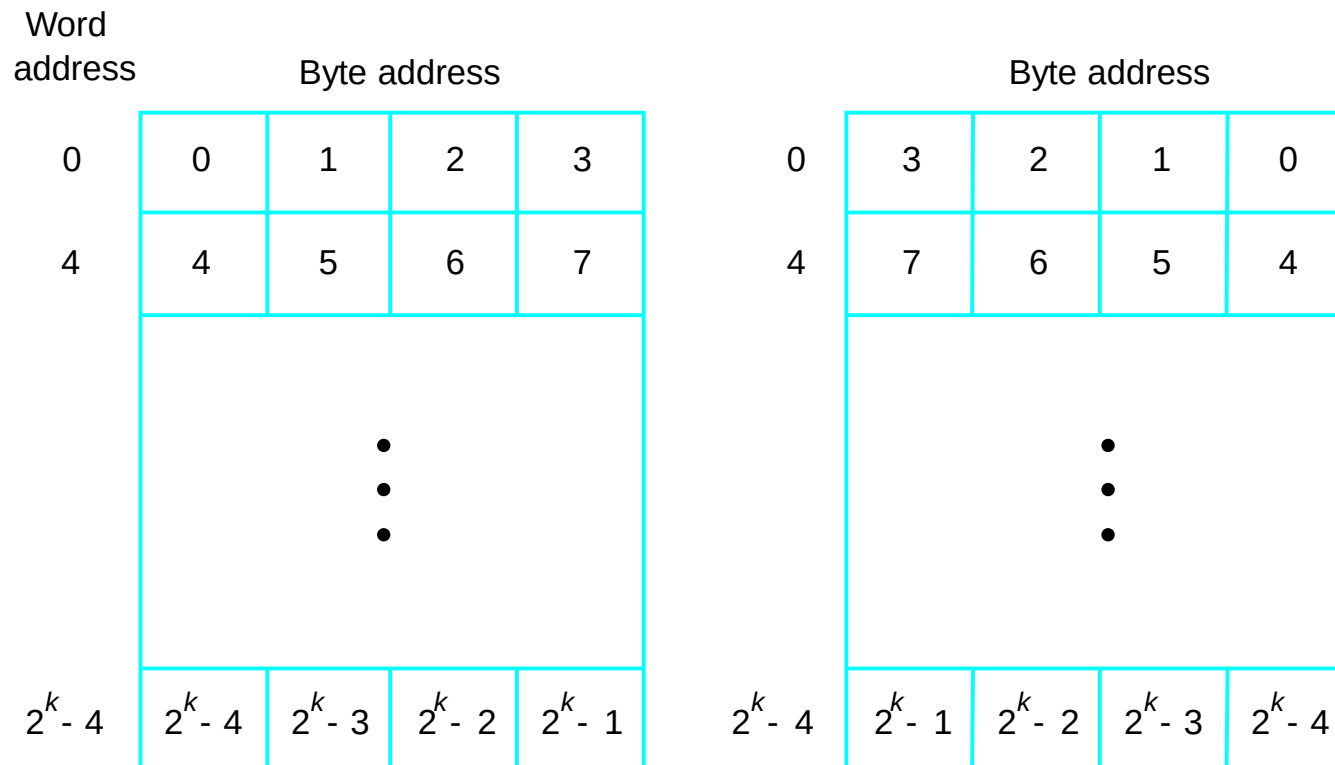
Memory Location, Addresses, and Operation

- It is impractical to assign distinct addresses to individual bit locations in the memory.
- The most practical assignment is to have successive addresses refer to successive byte locations in the memory – byte-addressable memory.
- Byte locations have addresses 0, 1, 2, ... If word length is 32 bits, then successive words are located at addresses 0, 4, 8,...

Big-Endian and Little-Endian Assignments

Big-Endian: lower byte addresses are used for the most significant bytes of the word

Little-Endian: opposite ordering. lower byte addresses are used for the less significant bytes of the word



(a) Big-endian assignment

(b) Little-endian assignment

Byte and word addressing.

Thank you